

Lab 6 – PageRank sobre Wikipedia usando Giraph

Big Data

30 de julio de 2026

Hoy vamos a usar PageRank para averiguar qué artículos de Wikipedia son “más importantes” y usar esos puntajes para mejorar el ranking de los resultados de búsqueda de la clase pasada (*idealmente has completado el Laboratorio 5 antes de este, pero en caso de que no lo hayas hecho, le dejaremos un método alternativo para que pueda completar este laboratorio*). Para calcular PageRank, usaremos Apache Giraph: un framework distribuido basado en Hadoop para procesar grafos.

Como entrada, se le proporciona un archivo TSV (valores separados por tabulaciones) con dos columnas. Este archivo codifica un grafo dirigido de enlaces entre artículos en Wikipedia. Más detalladamente, para cada par de artículos de Wikipedia A y B, existe un enlace $A \rightarrow B$ en el grafo si el artículo A contiene un enlace al Artículo B. Por ejemplo, las siguientes líneas en la entrada significan que el artículo de la izquierda enlaza con los tres artículos de la derecha:

http://es.wikipedia.org/wiki/Andorra	http://es.wikipedia.org/wiki/España
http://es.wikipedia.org/wiki/Andorra	http://es.wikipedia.org/wiki/Portugal
http://es.wikipedia.org/wiki/Andorra	http://es.wikipedia.org/wiki/Francia

Trabajaremos con el archivo `hdfs://cm:9000/uhadoop/shared/wiki/es-wiki-links.tsv.gz`, que contiene 31,506,565 líneas, cada una de las cuales representa un enlace de un artículo de Wikipedia a otro.

Primero codificaremos el algoritmo PageRank para estimar la importancia de los artículos en el grafo en función de cómo están enlazados. Luego utilizaremos estos valores para mejorar los resultados de búsqueda del Lab 5.

- Descargue el proyecto de código de u-cursos y ábralo en Eclipse. Su tarea inicial será completar la clase **PageRank**, que calcula los puntajes de PageRank sobre un grafo cargado en Giraph. Las otras clases están completas: **TextNullTextEdgeInputFormat** proporciona código para cargar el grafo en Giraph, mientras que **VertexValueTextOutputFormat** proporciona código para escribir los resultados de PageRank calculados por Giraph en un archivo TSV. **PageRankAgg** inicializa dos *aggregators* que pueden usarse para guardar cierta información global para el proceso de PageRank. Finalmente **SortByRank** es un trabajo de Hadoop que ordena descendientemente los resultados utilizando el PageRank (para que podamos ver los artículos más importantes al comienzo del archivo). Recomendamos revisar brevemente estas clases.
- Ahora para completar el algoritmo de PageRank. Abra la clase **PageRank**. La mayor parte del código está lista. Las constantes son las siguientes:
 - **ITERS** define el número de iteraciones de PageRank a ejecutar.
 - **D** es el factor de amortiguación: la probabilidad de que un surfista aleatorio siga un enlace cuando se encuentra en un nodo con enlaces salientes.
 - **RR_AGG_NAME** es un aggregator global que guarda la probabilidad de que el surfista aleatorio realice un salto aleatorio en la iteración anterior (ya sea desde un nodo colgante sin enlaces salientes o con probabilidad $1 - D$ de un nodo con enlaces salientes).
 - **NUM_V_NAME** es un aggregator global para contar la cantidad de vértices. **ONE** es un objeto que reutilizaremos para este recuento.

- El framework Giraph se basa en “superpasos”. En cada superpaso, cada vértice puede recibir mensajes del superpaso anterior y puede enviar mensajes para el superpaso siguiente. Dentro de cada superpaso, cada vértice llama al método `compute`. Un superpaso típico usará los mensajes entrantes de un vértice para actualizar su estado y luego calculará los mensajes salientes enviados desde el vértice a sus vecinos. Tenga en cuenta que la ID del primer superpaso es 0.
- Para calcular el PageRank, necesitamos saber el número de vértices, pero no lo tenemos *de antemano*. Si bien puede parecer trivial, dado que nuestro cálculo se basa principalmente en pasar mensajes entre vértices vecinos, contar todos los vértices no es tan directo. En su lugar, utilizaremos dos superpasos iniciales para contar los vértices en el grafo. El primer superpaso “activa” los vértices enviando mensajes ficticios a través del grafo. El segundo superpaso luego envía un 1 desde cada vértice a un aggregator global sum, que, como resultado, dará el recuento de vértices al final del superpaso.¹
- En el tercer superpaso, comenzamos la primera iteración de PageRank. Comenzamos obteniendo el recuento de vértices del aggregator e inicializamos un rank para el vértice. Si esta es la primera iteración, el rank del vértice se calculará como el recíproco del número de vértices ($\frac{1}{|V|}$). De lo contrario, si estamos en una iteración posterior, el rank del vértice se calculará como la suma de los mensajes entrantes de sus vecinos, y la probabilidad de salto aleatorio dividida por el número de vértices.
- Los vértices del grafo se pueden ver como nodos de cómputo que hacen cálculos. Si estamos en el último superpaso, le decimos a los vértices que se detengan y finalicen el cálculo. De lo contrario, si no estamos en el último superpaso, deberíamos enviar mensajes desde el vértice actual a sus vecinos para el paso siguiente, y actualizar los aggregators según sea necesario. En términos de qué mensajes debe enviar un vértice y cómo debe actualizar los aggregators, hay dos casos principales: el vértice actual tiene enlaces salientes o el vértice actual no tiene enlaces salientes. Te quedan tres TODO:
 - En el caso de que el vértice tenga enlaces salientes, debe (1) enviar mensajes para dividir una razón de D del rank actual del vértice de manera uniforme con sus vecinos, (2) enviar lo que queda del rank del vértice actual al aggregator para la probabilidad de salto aleatorio. Tenga en cuenta que en el caso de (1), `sendMessageToAllEdges` enviará el valor del argumento a cada uno de los vecinos del vértice.
 - En el caso de que el vértice no tenga enlaces salientes, (3) envíe el rank actual del vértice al aggregator para la probabilidad de salto aleatorio.
- Una vez que haya completado los tres valores antes mencionados, compile el JAR para el proyecto, transféralo a su carpeta personal en el clúster.
- Hemos proporcionado un pequeño ejemplo (`hdfs dfs -cat /uhadoop/shared/pr/pr-ex.tsv`) para realizar pruebas. Intente ejecutar su código en este pequeño ejemplo (el siguiente es un solo comando):²
 - `giraph gdd-giraph.jar org.mdp.hadoop.cli.PageRank`
`-eif org.mdp.hadoop.io.TextNullTextEdgeInputFormat`
`-eip /uhadoop/shared/pr/pr-ex.tsv`
`-vof org.mdp.hadoop.io.VertexValueTextOutputFormat`
`-op /uhadoop2026/cc66i/``GROUP-FOLDER``/pr-ex`
`-w 1 -ca mapreduce.job.tracker=yarn -ca mapreduce.framework.name=yarn`
`-mc org.mdp.hadoop.pr.PageRankAgg`

(Si obtiene un error de comando no encontrado, ejecute `source ~/.profile` e intente nuevamente.)

¹Giraph proporciona un método `getTotalNumVertices()`, pero esto devuelve `-1` en el primer superpaso, y parece no contar vértices colgantes sin enlaces salientes.

²Los argumentos indican, respectivamente, la clase con el método `compute`, el formato de entrada para leer los datos, el archivo de entrada, el formato de salida para escribir los datos, la carpeta de salida, el número de workers, dos configuraciones para ejecutar Giraph en Hadoop y, finalmente, la clase que inicializa los aggregators globales. Es bastante largo ya que no usamos un método `main`; más bien especificamos las configuraciones relevantes en la llamada al comando.

- Verifique la carpeta de salida:

- `hdfs dfs -ls /uhadoop2026/cc66i/[GROUP-FOLDER]/pr-ex/`

Eche un vistazo a los datos de salida que se encuentran dentro:

- `hdfs dfs -cat /uhadoop2026/cc66i/[GROUP-FOLDER]/pr-ex/part-m-0000[N]`

Los puntajes del PageRank deben sumar 1. El rank de “a” debe ser aproximadamente 0.169.

- Una vez que su código funciona, queremos ejecutarlo en los datos completos. Ejecute:

- `giraph gdd-giraph.jar org.mdp.hadoop.cli.PageRank
-eif org.mdp.hadoop.io.TextNullTextEdgeInputFormat
-eip /uhadoop/shared/wiki/es-wiki-links.tsv.gz
-vof org.mdp.hadoop.io.VertexValueTextOutputFormat
-op /uhadoop2026/cc66i/[GROUP-FOLDER]/pr-full
-w 4 -ca mapreduce.job.tracker=yarn -ca mapreduce.framework.name=yarn
-mc org.mdp.hadoop.pr.PageRankAgg`

Aumentamos el número de workers (**w**) a 4 para aumentar el paralelismo. Revise la carpeta de salida y los resultados dentro. Tenga en cuenta que el proceso puede tardar un tiempo en ejecutarse.

- Queremos saber cuáles son los artículos más importantes, ¿verdad? Para esto, utilizaremos un trabajo de Hadoop para ordenar los resultados por rank (descendente). Ejecutar (todo un comando): ³⁾

- `hadoop jar gdd-giraph.jar SortByRank -D mapreduce.job.reduces=1
/uhadoop2026/cc66i/[GROUP-FOLDER]/pr-full/ /uhadoop2026/cc66i/[GROUP-FOLDER]/pr-full-s/`

Mientras espera, intente adivinar cuáles podrían ser los artículos más importantes de Wikipedia. ¡Revise los primeros resultados para ver si está en lo correcto!

- ¿Acabamos de calcular el PageRank de los artículos de Wikipedia por diversión? Naturalmente sí. Pero ahora que los tenemos, también podemos mejorar los resultados de búsqueda para el Laboratorio 5. Idealmente, podríamos extender el código del laboratorio directamente, pero dado que el laboratorio aún está abierto, algunos aún no lo han completado. Del mismo modo, no podemos proporcionar el código porque incluye la solución para el Laboratorio 5 (publicaremos el código después de la fecha límite para el Laboratorio 5). Por el momento, proporcionamos un jar binario con el código compilado que puede usar. Lo más importante por ahora no es el código, sino cómo pueden cambiar los resultados de búsqueda cuando se incluye PageRank. (Como se mencionó en la clase, la relevancia y la importancia son a menudo medidas antagónicas y no hay una forma clara de combinarlas; aplicamos una combinación simple de ambos puntajes para una consulta que podría no funcionar muy bien para todas las búsquedas).

- Primero, necesitará tener los datos del ranking disponibles localmente. Llama (un solo comando):

- `hdfs dfs -copyToLocal /uhadoop2026/cc66i/[GROUP-FOLDER]/pr-full-s/part-r-00000
/data/2026/uhadoop/cc66i/[GROUP-FOLDER]/ranks.s.tsv`

- En el servidor, vaya a `cd /data/2026/uhadoop/cc66i/lab5`. Encontrarás un jar. Llama (mejor usar un nombre en **minúscula** para [NEW-INDEX-NAME] porque a veces Elasticsearch no reconoce nombres en mayúscula):

- `java -jar gdd-elastic.jar BuildWikiIndexBulkWithRank
-i /data/uhadoop/shared/wiki/es-wiki-articles.tsv.gz -igz
-p /data/2026/uhadoop/cc66i/[GROUP-NAME]/ranks.s.tsv -o [NEW-INDEX-NAME]`

³La propiedad `-D mapreduce.job.reduces=1` establece el número de reducers en uno, por lo que obtenemos un archivo de salida.

Esto indexará los mismos documentos que Lab 5, pero ahora con nuestros puntajes de PageRank como un nuevo campo.

- Usando `gdd-elastic.jar`, ejecute `SearchWikiIndex` and `SearchWikiIndexWithRank` sobre el nuevo índice para realizar búsquedas y comparar los efectos sin (`SearchWikiIndex`) y con puntajes de PageRank (`SearchWikiIndexWithRank`):

- `java -jar gdd-elastic.jar SEARCH-CLASS -i NEW-INDEX-NAME`

En particular, `SearchWikiIndexWithRank` recuperará el puntaje de PageRank y lo combinará con el puntaje de relevancia para reordenar los resultados, teniendo en cuenta la importancia del artículo. `SearchWikiIndexWithRank` debería, por otro lado, dar los mismos resultados que Lab 5, sin considerar el PageRank.

Intente buscar “`trump`” con y sin PageRank. Pruebe otras cinco búsquedas de su elección. ¿Cómo cambian los resultados?

- ENTREGA: (1) su clase `PageRank.java`, (2) los 10 mejores artículos de Wikipedia según PageRank, y (3) los resultados de cinco consultas (una de las cuales debería ser “`trump`”) con y sin agregar PageRank.